■ AUTOMATISATION CI/CD AVEC DOCKER

Mini-Projet DevOps • Everything as Code

Présenté par	Encadrant
Anas Haddou Amine Mohammed Tayek	Prof. Mohtaram

57s	100%	89%	30ms
Durée pipeline	Tests passés	Réduct. image	Temps réponse

École Marocaine d'Ingénierie • Rabat

Ingénierie des Systèmes d'Information • 2024–2025

1 Introduction & Contexte

Philosophie DevOps • Everything as Code

Ce projet illustre la mise en œuvre concrète d'un pipeline d'intégration et de déploiement continus (CI/CD) complet, depuis l'écriture du code jusqu'au déploiement automatisé en production. L'objectif central est d'**éliminer les interventions manuelles**, de réduire les délais et d'assurer une livraison fiable et reproductible.

Concept	Description
DevOps	Convergence Développement + Opérations → cycles courts, qualité élevée
Everything as Code	Infrastructure, config, pipelines → tous versionnés dans Git
CI (Intégration Continue)	Tests automatisés à chaque commit pour détecter les régressions
CD (Déploiement Continu)	Mise en production automatique si tous les tests passent

2 Objectifs & Stack Technique

Node.js • Docker • GitHub Actions • Docker Hub

■ Objectifs du projet

■	**Développer une API web robuste**	`Node.js + Express`
■	**Conteneuriser de façon optimale**	`Docker Multi-Stage`
■	**Automatiser le cycle de vie complet**	`GitHub Actions workflow`
■	**Couverture de tests 100%**	`Jest + Supertest sur endpoints critiques`
■	**Réduire la taille de l'image de 89%**	`Alpine Linux base image`

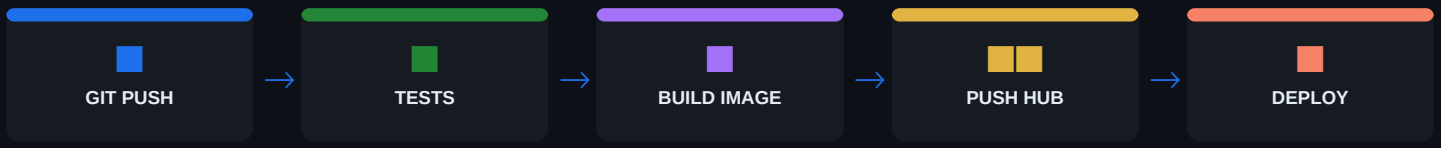
■■ Stack Technique

Couche	Technologie	Rôle
Application	Node.js + Express	API REST, serveur web
Tests	Jest + Supertest	Tests unitaires & intégration automatisés
Conteneurisation	Docker + Alpine	Image légère 100 Mo (vs 900 Mo standard)
CI/CD Engine	GitHub Actions	Orchestration du workflow automatisé
Registre	Docker Hub	Stockage sécurisé des images
Déploiement	Docker Compose	Orchestration locale des conteneurs
Observabilité	Dashboard DevOps	Santé, uptime, temps de réponse en temps réel

3 Architecture du Pipeline CI/CD

Push → Test → Build → Push → Deploy (57 secondes)

Le pipeline est déclenché automatiquement dès qu'un développeur effectue un **git push** sur la branche principale. Chaque étape est conditionnée à la réussite de la précédente : si les tests échouent, la construction ne démarre pas.



1. Git Push (Déclencheur)

Le développeur pousse son code sur la branche main. GitHub détecte l'événement push et déclenche immédiatement le workflow .github/workflows/ci-cd.yml.

2. Exécution des Tests

GitHub Actions installe les dépendances Node.js et lance Jest + Supertest. L'étape suivante ne s'exécute QUE si tous les tests passent à 100%.

3. Construction de l'Image Docker

Le Dockerfile Multi-Stage est utilisé : la première étape compile/installe, la seconde copie uniquement les fichiers nécessaires sur Alpine. Résultat : 100 Mo au lieu de 900 Mo.

4. Push vers Docker Hub

L'image construite est taguée et poussée vers Docker Hub en utilisant les GitHub Secrets pour l'authentification (aucun mot de passe en clair).

5. Déploiement Automatique

Le serveur récupère la nouvelle image via docker pull et la déploie avec Docker Compose. Le service redémarre sans interruption visible.

Structure du workflow GitHub Actions

```yaml
# .github/workflows/ci-cd.yml
name: CI/CD Pipeline

on:
  push:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - run: npm install
      - run: npm test

  build-and-push:
    needs: test
    steps:
      - uses: docker/login-action@v2
        with:
          username: ${{ secrets.DOCKER_USERNAME }}
          password: ${{ secrets.DOCKER_TOKEN }}
      - run: docker build -t monapp:latest .
      - run: docker push monapp:latest

  deploy:
    needs: build-and-push
    steps:
      - run: docker compose up -d --pull always
```

4 Optimisations & Sécurité

Multi-Stage Build • Alpine • GitHub Secrets

■ Optimisation Docker — Multi-Stage Build

Le principal défi était la taille des images Docker. Une image Node.js standard pèse environ **900 Mo**. Grâce au **Multi-Stage Build** avec une base Alpine, l'image finale ne pèse plus que **100 Mo** — une réduction de **89%**.

Image	Taille	Gain
node:18 (standard)	~900 Mo	Référence
node:18-alpine (Multi-Stage)	~100 Mo	-89% ■

```
# Dockerfile – Multi-Stage Build

# ■■ Étape 1 : Builder ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
FROM node:18 AS builder
WORKDIR /app
COPY package*.json ./
$ npm install --production
COPY . .

# ■■ Étape 2 : Image finale légère ■■■■■■■■■■■■■■■■■■■
FROM node:18-alpine
WORKDIR /app
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app .
EXPOSE 3000
CMD ["node", "server.js"]
```

■ Sécurité — GitHub Secrets

Lors des premiers tests, l'authentification Docker Hub échouait car les credentials étaient exposés en clair dans le code. La solution : les **GitHub Secrets** chiffrent toutes les informations sensibles.

Secret	Contenu	Utilisation
DOCKER_USERNAME	Nom d'utilisateur Docker Hub	docker/login-action dans le workflow
DOCKER_TOKEN	Personal Access Token (PAT)	Authentification sécurisée sans MDP

5 Dashboard DevOps & Observabilité

Santé • Uptime • Temps de réponse • Métriques temps réel

Pour une visibilité complète sur l'état du système, un **Dashboard DevOps** accessible sur **http://localhost:3000/** affiche en temps réel les métriques critiques de l'application.

■ UP	99.9%	~30ms	100%
Statut Système	Uptime	Temps de réponse	Tests passés

■ Endpoints surveillés

Endpoint	Méthode	Description	Couverture
`/health`	GET	Vérifie que l'application est vivante	■ 100%
`/metrics`	GET	Métriques uptime et temps de réponse	■ 100%
`/api/*`	GET/POST	Endpoints métier de l'application	■ 100%

6 Guide de Démonstration Live

Déroulé étape par étape pour la présentation

ÉTAPE 01	**Montrer le Dashboard — Anas** Ouvrir http://localhost:3000/ dans le navigateur. Montrer le statut UP, l'uptime, le temps de réponse (~30ms) et les métriques. Expliquer que tout est en production réelle.
ÉTAPE 02	**Modifier le code — Amine** Dans l'éditeur, changer la version de l'application ou un texte visible sur le dashboard. Ce changement sera visible après le déploiement automatique.
ÉTAPE 03	**Git Push — Anas** Exécuter : git add . → git commit -m 'demo: update version' → git push. Le pipeline se déclenche immédiatement sur GitHub Actions.
ÉTAPE 04	**Suivre le pipeline — Amine** Ouvrir l'onglet Actions du repository GitHub. Commenter chaque étape en direct : Tests ■ → Build ■ → Push ■ → Deploy ■. Durée totale ≈ 57s.
ÉTAPE 05	**Vérifier le résultat — Anas** Rafraîchir le Dashboard. La modification est instantanément visible. Preuve que le pipeline fonctionne de bout en bout sans intervention manuelle.

7 Conclusion & Perspectives

Résultats atteints • Prochaines étapes

■ Résultats obtenus

Pipeline complet et fonctionnel	Test → Build → Push → Deploy en 57s
Everything as Code	Reproductibilité totale de l'environnement via Git
Sécurité assurée	Aucun credential exposé — GitHub Secrets uniquement
Performance optimisée	Image Docker réduite de 89% (900 Mo → 100 Mo)
Observabilité en temps réel	Dashboard avec santé, uptime et latence

■ Perspectives d'évolution

Technologie	Apport
Kubernetes	Orchestration automatique des conteneurs à grande échelle
SonarQube	Analyse statique de la qualité et de la sécurité du code
Prometheus + Grafana	Observabilité avancée avec dashboards et alerting
GitOps (ArgoCD)	Déploiement piloté par Git pour encore plus d'automatisation

Merci pour votre attention !

Nous sommes à votre disposition pour toute question.